

Vigil-X Solution Architecture Document



Document Details

Version	1.1
Document Date	17-June-2025
Author(s)	Vijay Agavinti
Reviewer(s)	Arvind, Gauri
Prepared For	Vigil-X
Confidentiality Level	Internal

Table of Contents

Architecture Overview:	4
Server Setup:	6
Hardware Specifications and Load Capacity	7
Solution Architecture Diagram:	8
1. Media Server:	10
Ingestion Workflow	10
Live Monitoring Distribution	12
Video Segmentation and Upload	14
AI Forwarding	14
Bitrate and FPS Control	14
High Availability Setup	15
Media Server Threshold and Overuse	16
2. Storage, Search, and Playback:	16
I. Recording and Storage	16
Segmentation and Streaming	16
Encoding and Processing	17
Storage and Indexing	17
Metadata Generation and Indexing	18
Storage Backend and Upload Strategy	18
Retention and Archival Policy	19
II. Search and Playback	19
Search API	19
Segment Resolution and Access	20
3. AI Service and Event Management:	20
I. Stream Routing and Inference Workflow	20
Stream Ingestion and Routing	20
Inference Pipeline Execution	20
II. Event Detection, Asset Generation, and Metadata Handling	21
Real-Time Event Processing	21
Metadata Indexing	21
4. Non-Core Modules:	22
4.1 Notification and Event Dispatcher	22
Workflow and Processing	22
Features and Optimizations	22
4.2 Admin Services	22
Workflow and Data Flow	22
Features and Optimizations	23
4.3 Auto-Discovery Service	23
Discovery Workflow	23
Features and Optimizations	23

4.4 Audit and Monitoring.....	23
Logging and Monitoring Pipeline.....	24
Features and Optimizations.....	24
4.5 Health Monitoring.....	24
Health Check Architecture.....	24
Features and Optimizations.....	24
4.6 Authentication (Auth Service).....	24
Authentication Flow.....	25
Features and Optimizations.....	25
4.7 Password and Secrets Management.....	25
4.8 Event Management.....	25
Stream Processing and Event Detection.....	26
Event Asset Generation.....	26
Storage and Indexing.....	26
Notification Pipeline.....	26
4.9 Communication with External Devices.....	27
4.10 Handling External VMS Systems.....	28
WebRTC Security Summary.....	28
1. Encryption.....	28
2. Authentication & Identity.....	29
3. Access Control.....	29
4. Signaling Security.....	29
5. Privacy Concerns.....	29
6. Mitigating Attacks.....	29
7. Regulatory Compliance of Indian government and System certificates.....	30
Multi-Cloud Deployment.....	30
Goals.....	30
Architecture Options.....	30
1. Hub-and-Spoke Model.....	30
2. Provider-Agnostic Deployment.....	30
3. Failover Model.....	31
Use Cases to Architecture Component Mapping.....	31
Relevant Links	
Question and answers doc:	
Arch patterns doc:.....	32

Vigil-X Solution Architecture

Architecture Overview:

The Vigil-X Video Management System (VMS) is designed to handle large-scale, multi-site surveillance needs by integrating edge-level media processing with a centralized event and user management platform. The system begins at the input layer, where video streams are sourced from various devices such as IP cameras, edge cameras, or NVRs. These devices transmit RTSP streams, which are ingested at the respective site-level deployment.

Each site includes a dedicated VigilX-Media-Server responsible for handling the full media processing pipeline locally. Upon receiving the RTSP feed, the media server performs parallel processing tasks including stream replication for live view over WebRTC, real-time video analytics for threat or anomaly detection, and continuous health monitoring of each stream. The media server also segments the incoming stream into 10- to 30-second chunks (configurable) using FFmpeg, stores them in local hot storage such as MinIO or S3 Standard, and writes associated metadata such as timestamps and event labels to TimescaleDB. After a 24-hour retention period, the data is transitioned to cold archival storage such as S3 Glacier or an equivalent deep-storage solution to optimize cost efficiency. Additionally, smart recording optimizes quality with AI-powered scene detection, and bandwidth-aware recording adjusts quality based on network conditions to enhance performance and storage efficiency.

The media server includes an auto-discovery service that scans the local network to detect and register all available camera devices using ONVIF or similar protocols. When the AI analytics pipeline detects an event of interest—such as motion, intrusion, or object classification—the system generates a snapshot and pre/post-event video segment enriched with timestamped metadata and classification labels. These are sent to the alert management service. The alert is then passed to a message dispatcher for further processing and routing.

The site also hosts an NGINX and HAProxy-based API Gateway that facilitates low-latency access to live video streams and alerts via desktop clients or video walls located within the same network.

All site-level events and alerts are streamed to the central Vigil-X Center through a Kafka-based messaging pipeline, enhanced with Apache Kafka for high-throughput event streaming. At the central layer, these events are consumed by an event management module that performs classification (true alert, false positive, etc.) and stores the structured event data

in MongoDB. The validated events are then routed to a notification dispatcher, which sends push notifications to authorized users and devices across web and mobile applications. Adaptive streaming for mobile viewing adjusts quality based on device capabilities and connection speed, ensuring optimal performance.

The central system includes a search and playback module that allows authorized users to retrieve and view archived footage from any site. This module interacts with metadata indexed in PostgreSQL and cached in a Redis Cluster for efficient querying and playback, with ClickHouse alongside TimescaleDB optimizing analytics queries. Intelligent tiering automates footage movement to cheaper cloud storage (e.g., S3 Glacier), predictive analytics alerts on storage needs, and multi-cloud support (AWS, Google Cloud, Azure) ensures redundancy.

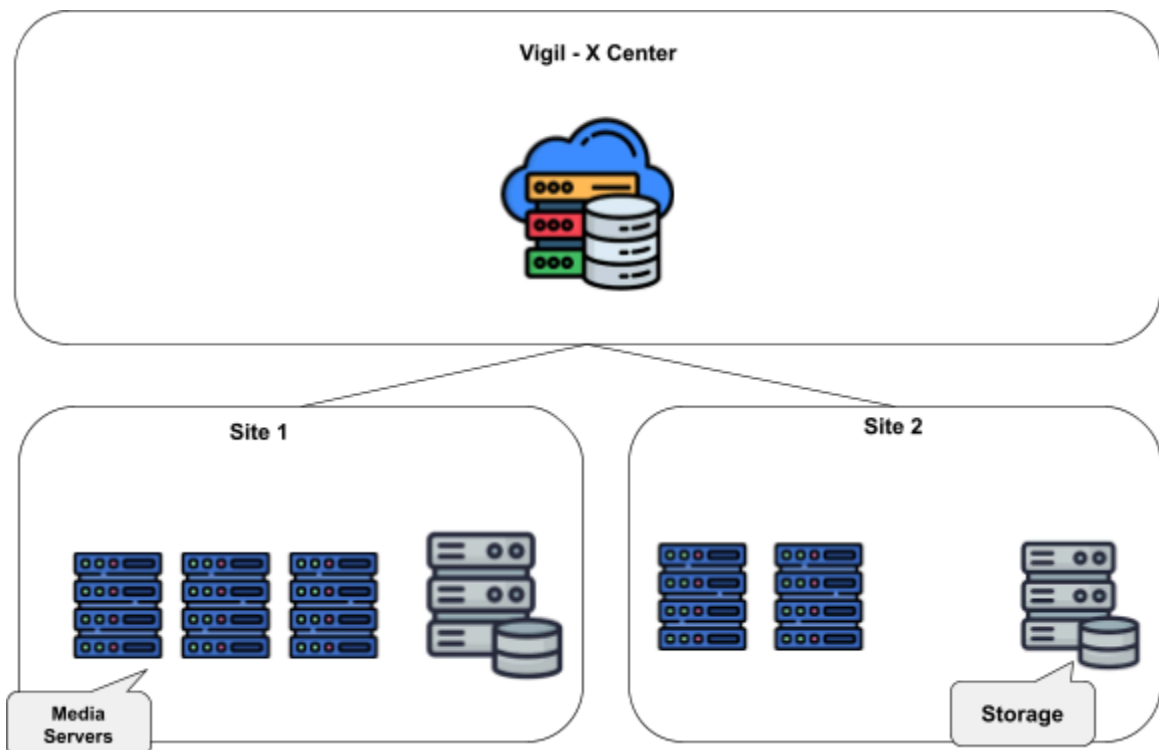
The central control layer contains administrative services responsible for managing users, devices, sites, and customer onboarding, now implemented using Django REST Framework for robust user management, permissions, and admin interfaces leveraging its ORM. FastAPI is retained for high-performance video streaming endpoints. A proof-of-concept evaluation is planned for gRPC to optimize internal AI communication and Flask for lightweight notification microservices. These services operate on a PostgreSQL database, with caching optimized by Redis.

A dedicated health monitoring module tracks the operational status of both central and site-level services, augmented by Prometheus, Grafana, and Jaeger for enhanced observability. Device Management Service (Django), Video Analytics Service (FastAPI + gRPC), Notification Service (Flask), Reporting Service (Django + Pandas), and Configuration Service (FastAPI)—are orchestrated via Kubernetes, improving modularity.

To ensure scalability, resilience, and performance at scale, the Vigil-X architecture is fully containerized using Docker and orchestrated via Kubernetes. Each microservice, including the media server, analytics modules, message dispatchers, and admin APIs, runs as an isolated container, allowing independent scaling based on workload demands. The media server remains stateful, deployed with persistent volumes and managed through Helm charts and StatefulSets, ensuring high availability and data durability. Load balancers and ingress controllers route traffic efficiently to API Gateways and core services, while monitoring tools provide visibility into system health. The architecture supports horizontal scaling across cloud and on-premise environments, handling tens of thousands of cameras without compromising reliability, performance, or operational manageability. CDN integration supports global video delivery, and database read replicas enhance query performance.

Server Setup:

The Vigil-X server infrastructure is designed to operate across multiple zones, with each zone functioning as an independent unit capable of handling a large number of video streams. Each camera can output up to three streams: one for storage, one for live monitoring and AI processing, and a third optionally dedicated to AI-based alert generation. These streams are managed locally by **Vigil-X Media Servers** deployed at each site. The media servers are horizontally scalable—additional instances can be deployed dynamically within the same site as camera load increases. Each media server handles stream ingestion, AI analytics, video chunking, and communication with internal site-level services.



At the zone level, three key server types are deployed: **Storage Servers** for handling hot and cold video data, **Live Monitoring Servers** to deliver real-time feeds to local clients, and **AI Servers** that process live streams for analytics and alert detection. These servers are deployed on-premises or in edge data centers based on the geographic location and network constraints of the zone. All intra-zone traffic is routed over secure LAN with VLAN segmentation, while inter-zone and central communication—including alerts and metadata.

Server capacity within a zone scales automatically—when CPU, memory, or GPU utilization crosses defined thresholds (e.g., 70%), the system triggers deployment of additional media,

AI, or processing nodes. This ensures consistent performance even during peak usage. The design also incorporates failover readiness via active-passive configurations for critical services within each zone to maintain high availability.

The setup supports any number of zones, each configured uniformly, allowing Vigil-X to grow linearly without bottlenecks. This modularity and zone-level independence enable robust, high-throughput, and low-latency video management suitable for deployments ranging from small sites to nationwide networks.

The deployment strategy includes new microservices—Device Management Service (Django), Video Analytics Service (FastAPI + gRPC), Notification Service (Flask), Reporting Service (Django + Pandas), and Configuration Service (FastAPI)—orchestrated via Kubernetes. CDN integration supports global video delivery, and database read replicas enhance query performance.

Hardware Specifications and Load Capacity

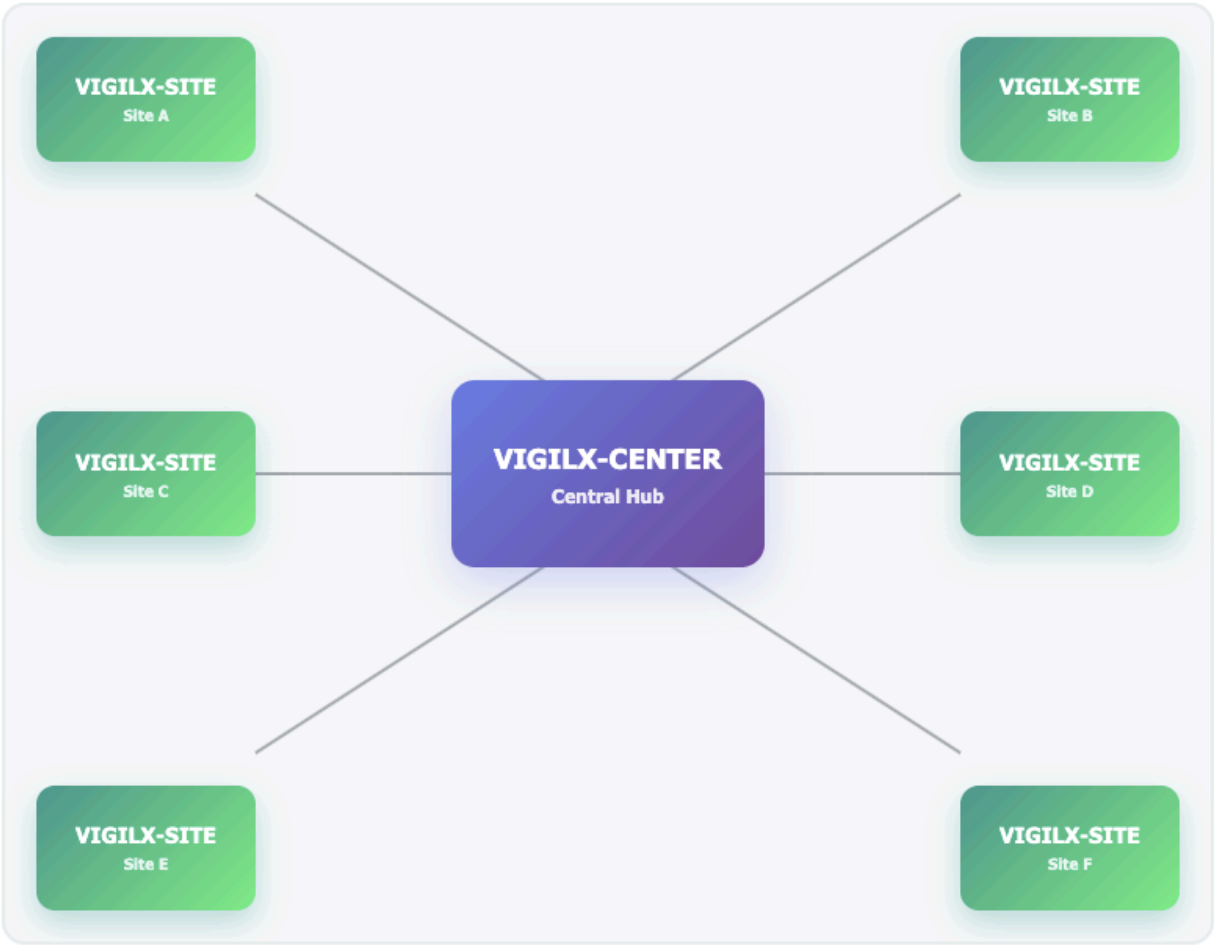
- **Without AI and Transcoding:**

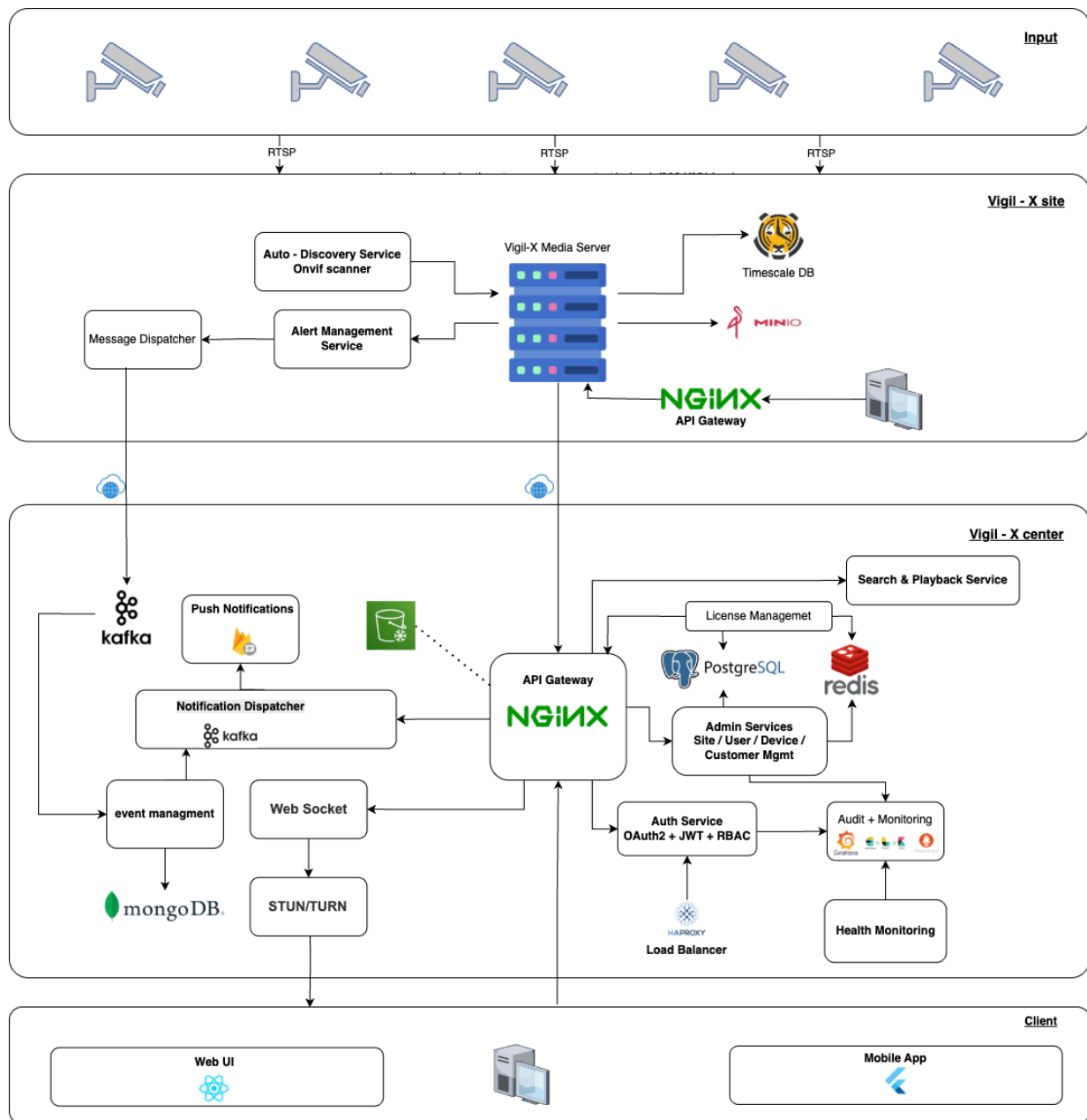
- **Specs:** 4-core CPU (e.g., Intel Xeon), 16 GB RAM, 480 GB .M2 SSD, and a 10–20 Gbps network interface card.
- **Load:** Handles ~1024 cameras streaming at 500 Kbps (H.264), with each camera using ~1.1 Mbps (video, audio, control data). (both inbound and outbound)
- **Key Calculations:**
 - **Network:** 1024 cameras * 1.1 Mbps = ~1.125 Gbps, which fits within a 10–20 Gbps NIC.
 - **Memory:** Each camera needs ~12 MB buffer, so 1024 cameras use ~12.3 GB, leaving room for 16 GB RAM for the OS and services.
 - **Storage:** Video segments (30s) write at ~70 MBps. A 480 GB .M2 SSD handles this but limits retention to ~1–2 hours before moving data to MinIO S3 storage.
- **Note:** Works best in a low-jitter (<10 ms) LAN with optimized settings.

- **With AI and Decoding:**

- **Specs:** Add a 16 GB NVIDIA RTX GPU (e.g., RTX 4000) to the media server.
- **Load:** Supports ~20–30 AI streams at 720p for YOLOv8 object detection.
- **Key Calculations:**
 - **GPU Memory:** Each stream uses ~0.5–1 GB, so a 16 GB GPU handles ~20 streams with batch processing.
 - **Inference Time:** ~20–40 ms per frame at 720p with optimization.
- **Note:** At 1080p or with complex models (like facial recognition), capacity drops to ~10–15 streams. Edge AI cameras can offload work to ease GPU load.

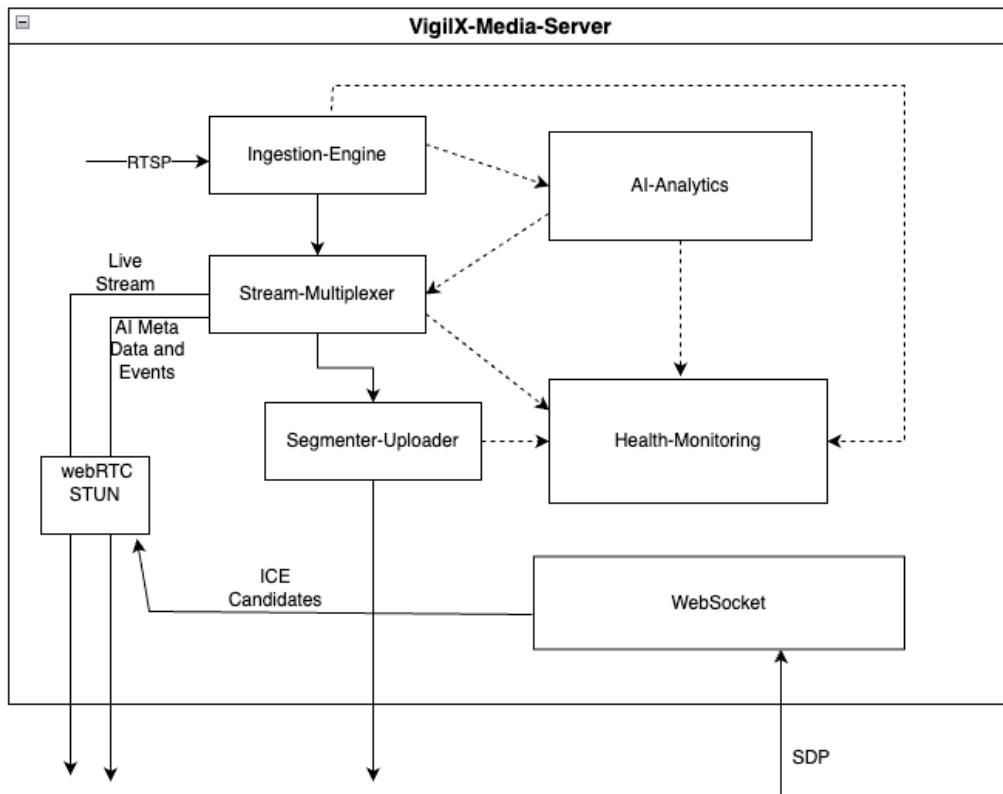
Solution Architecture Diagram:





1. Media Server

The Media Server is a self-contained processing unit deployed at each site to handle camera stream ingestion, live monitoring distribution, optional AI stream forwarding, and segment-based recording. It ensures all critical operations are localized at the site level to reduce network dependency, enable faster processing, and simplify scaling.



Ingestion Workflow

The Media Server begins by ingesting RTSP streams from IP cameras. Each camera typically provides a primary stream (used for recording), a secondary stream (used for live monitoring and/or AI), and optionally a third stream (reserved for high-resolution AI alerting). These streams are pulled using a **GStreamer/FFmpeg-based ingestion engine**.

After streams are passed through modules like webrtc to convert them into delivery-ready packets SRTP. Stream health is continuously monitored for packet loss, disconnections, or latency spikes. If an issue is detected, the engine raises a real-time alert using **WebSocket connections**, allowing clients or admins to respond immediately.

Parameter of Media Server	WebRTC	RTSP
Protocol Layer	ICE, DTLS, SRTP	RTP/RTCP
Transport Protocol	UDP/TCP with STUN/TURN, DTLS-SRTP	RTP/RTCP over UDP or interleaved over TCP
Media Description	SDP (offer/answer via signaling)	SDP (via DESCRIBE response)
Media Control	Peer-to-peer with signaling wss and https	RTSP commands: Describe, Setup, Play and Teardown
Encryption	DTLS-SRTP (mandatory)	None
Authentication	Application-specific (JWT, OAuth, API tokens)	Basic/Digest Auth (RFC 2617)
NAT Traversal	STUN / TURN / ICE	None built-in
Firewall Compatibility	High (TURN fallback, multiplexed RTP/RTCP)	Low (ports often blocked by firewalls)
Codec Support	H.264/H.265, Opus, G.711	H.264/H.265, AAC, G.711 (depends on camera/server)
Security Parameters	SSL, fingerprints	Session control and authorization
Media Sync (SSRC)	Yes	Not commonly used
Security by Design	End-to-end encrypted and verified	Security is optional and weak unless wrapped in TLS
BWE	500kbs-1mbs per stream	500kbs-1mbs per stream
I P B not supported	Base & Main profile, I size 500kps 100-200 B	Base & Main profile, I size 500kps 100-200 B
Congestion Control	Transport-cc and Meld	No proper algo. Mostly done with RTCP SR RR

Live Monitoring Distribution

For local viewing at the site, the decoded secondary stream is forwarded to a **WebRTC-based distribution layer** powered by **SFU**. This component handles stream multiplexing and delivers a single decoded stream to multiple concurrent viewers without duplication. To ensure connectivity across various network environments, **STUN and TURN services** are provided using **coturn**, which supports NAT traversal and session relay fallback. Adaptive streaming for mobile viewing adjusts quality dynamically based on device capabilities and network conditions, enhancing performance across diverse clients.

As part of the WebRTC session establishment, the system handles **ICE candidate exchange** and session negotiation with clients. This enables reliable video delivery across firewalls, enterprise NATs, or mobile networks without user-side configuration.

WebRTC Flow: External Client to On-prem Media Server

Step-1: Device Discovery (Control Plane via https)

- Client authenticates via VigilX-Center
- Gets access to: site list, available cameras & WebRTC signalling endpoint

Step-2: Stream request via websocket signalling

- Client connects via WebSocket
- Sends stream start request

Step-3: Routing to correct on-prem site

- Over the pre-established WebSocket bridge.
- The VigilX-Media-Server is now aware that a new viewer wants the stream.

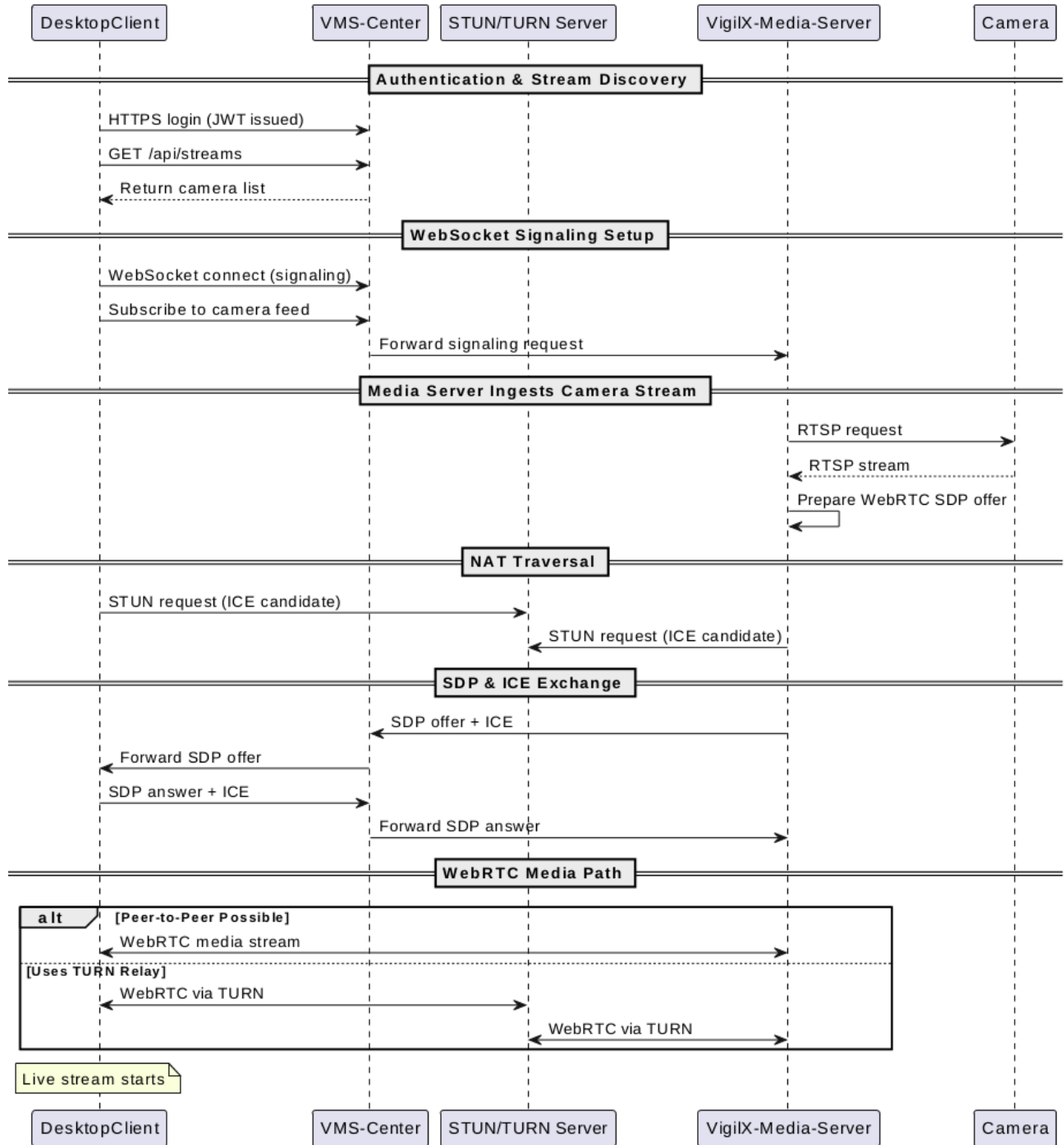
Step-4: SDP exchange and ICE setup

- Desktop client performs STUN lookup to get its public IP
- Media server (on-prem) also runs ICE negotiation

Step-5: Media Flow Establishment (Peer-to-Peer or Relayed)

- If client and on-prem server can connect:
 - Direct UDP WebRTC stream is established.

WebRTC Stream: Remote Desktop Client to VigilX-Media-Server in the VMS-Site



Video Segmentation and Upload

Simultaneously, the primary stream is directed to the **Segmenter-Uploader**, where it is processed for long-term storage. This component uses fmp4 mdat boxes the video into segments, typically 10–30 seconds(configurable) in length. The encoded segments are coupled with metadata like stream ID and timestamps. Once ready, the video segments and their metadata are published to the internal **Kafka pipeline**, where they are picked up by storage and indexing services outside the Media Server module. This asynchronous push model ensures fault tolerance and avoids bottlenecks during high-load scenarios.

AI Forwarding

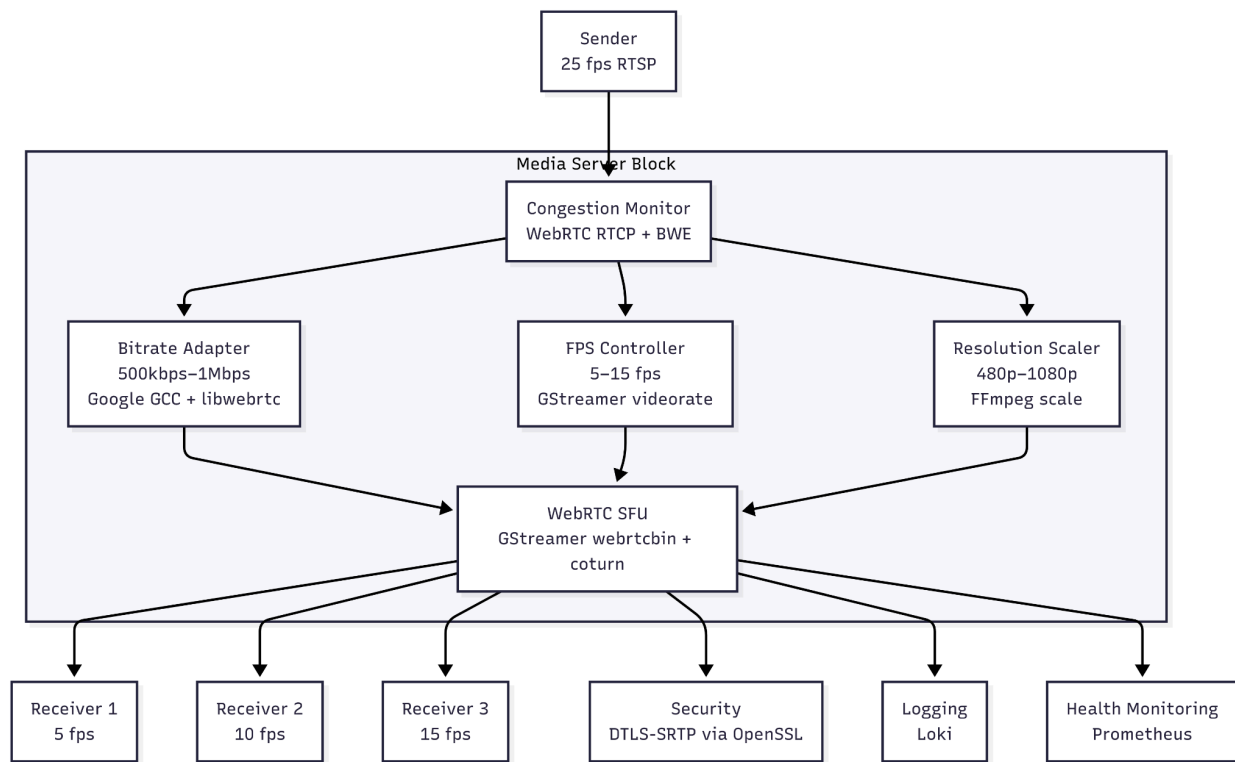
If AI-based analytics are enabled at the site, the Media Server forwards to the local AI engine for inference. This forwarding occurs independently of live viewing and recording workflows and is implemented over Kafka or shared memory pipes based on deployment type. The AI engine processes frames and publishes event metadata — such as object classes, bounding boxes, and timestamps — back into the system. This metadata is consumed by the client UI over WebSocket or REST, enabling real-time overlay of analytics results (e.g., intrusion zones, detected persons, vehicles) directly on the live video feed. This architecture ensures that users can view live AI annotations without affecting the core video stream performance. Stream processing includes smart recording with AI-powered scene detection and bandwidth-aware recording, adjusting quality based on network conditions to improve performance and storage efficiency.

Bitrate and FPS Control

The stream flow begins with a 25 fps RTSP (Real-Time Streaming Protocol) stream from the sender, ingested via GStreamer (rtspsrc) and buffered through a decoding queue. Frame types such as I, P, and IDR (Instantaneous Decoder Refresh) are handled during encoding and rate control for keyframe prioritization.

The Congestion Monitor continuously analyzes network conditions (e.g., packet loss, round-trip time) using WebRTC RTCP feedback and transport-cc bandwidth estimation approximately every 100 ms. This telemetry is used to dynamically adjust stream parameters via the following adaptive control modules:

- **Bitrate Adapter:** Modifies bitrate between 500 kbps and 1 Mbps using Google Congestion Control (GCC) in libwebrtc, optimizing for fluctuating bandwidth conditions.
- **FPS Controller:** Reduces frame rate using GStreamer videorate, maintaining keyframes (I/IDR) to preserve visual quality during congestion.
- **Resolution Scaler:** Downscales video resolution to 480p, 720p, or 1080p using the FFmpeg scale filter, based on client/network capability.



The adapted stream is then forwarded to the WebRTC SFU (Selective Forwarding Unit) implemented with GStreamer webrtcbin and coturn (for STUN/TURN support), which delivers a single encoded stream to multiple receivers (e.g., at 5 fps, 10 fps, 15 fps) depending on their capabilities.

The entire media pipeline is secured with DTLS-SRTP encryption via OpenSSL, ensuring end-to-end protection of video and control channels. In parallel, Loki captures logs for auditing and operational visibility, while Prometheus collects system health and stream quality metrics for proactive monitoring and alerting.

High Availability Setup

- A standby secondary media server is kept per site for failover.
- Health check service keeps pinging the primary media server every one second. In case of health check failure, it will call an API to re-register the active streams (available from Redis cache) on the secondary media server.
- Viewer clients should retry if disconnected.
- Both primary and secondary media servers should write to the shared MinIO & TimescaleDB.
- Site to media servers mapping is stored in the DB and Redis cache.

- Media server to active streams mapping is maintained in the Redis cache and is updated by the Health Monitoring service.

Media Server Threshold and Overuse

Thresholds: Alerts trigger when CPU or GPU usage exceeds 70%, memory exceeds 80%, or disk IOPS exceeds 90%. Kubernetes steps in to scale resources at these levels.

Overuse Handling:

- **Autoscaling:** Kubernetes spins up new media server pods (e.g., 4 extra cores, 16 GB RAM) when thresholds are crossed.
- **Failover:** If a server fails, active-passive failover kicks in, with Redis preserving sessions and MinIO/S3 syncing data to a standby zone.
- **Health Monitoring:** A HealthChecker microservice checks stream latency (>500 ms), packet loss (>5%), and resource usage every 20 seconds, notifying admins via WebSocket alerts. (check streams are coming or not)
- **Note:** For 1024 cameras, 4-core CPU hits ~70% without AI, and overuse prompts extra pods. With AI, GPU overuse (>70%) triggers new GPU nodes or offloading to edge AI.

2. Storage, Search, and Playback

I. Recording and Storage

The Recording and Storage component manages the segmentation, indexing, and upload of live video streams from cameras. It operates independently at each site to ensure low-latency processing and minimize reliance on centralized services, enabling efficient video segment storage with associated metadata for time-based search and playback.

Segmentation and Streaming

Video streams are streamed from cameras to the Media Server using RTSP over ONVIF Profiles G/S/T. The Media Server checks for the presence of an SD card in the camera. If an SD card is present, it verifies the SD card's health using SMART metrics and retrieves any missed video segments. If no SD card is present, it logs missing segments for notification.

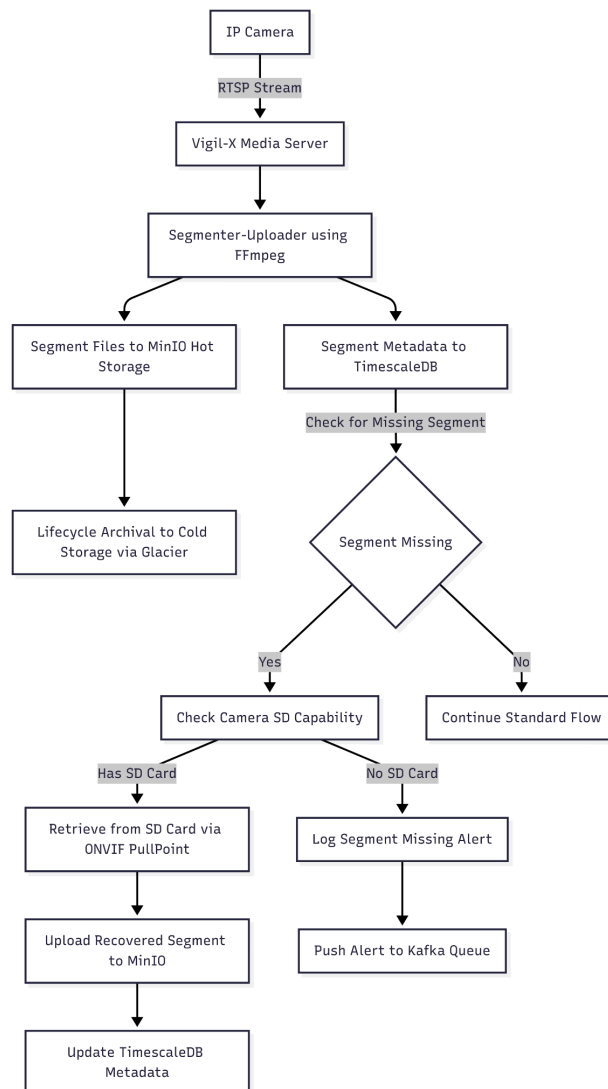
Encoding and Processing

All incoming streams, whether from the camera or SD card, are processed by the Recording Service using FFmpeg. The streams are decoded and segmented into 10–30-second chunks using FFmpeg's `-segment_time` option and encoded with libx265 for storage efficiency.

Segmentation and encoding leverage an async worker queue with parallel threading to ensure high-throughput recording. Smart recording employs AI-powered scene detection to optimize video quality, and bandwidth-aware recording adjusts quality based on network conditions to enhance performance and storage efficiency. Each video segment is uniquely named with a predictable format incorporating camera ID and timestamp.

Storage and Indexing

Segments are temporarily buffered in RAM or SSD, encrypted with AES-256, and uploaded to hot storage (MinIO or NAS). Metadata for each segment, including camera ID and timestamps, is indexed in TimescaleDB to support efficient search and playback. Segments are retained in hot storage for a configurable period (24 hours to 90 days) before transitioning to cold archival storage (S3 Glacier or on-prem archival).



Metadata Generation and Indexing

For every segment, structured metadata is generated, which includes:

- `cam_id`
- `start_time` and `end_time`
- `segment_filename`
- `storage_path`
- `event_type` or `motion_flag` (optional)

This metadata is written into TimescaleDB, a time-series optimized database. Hypertables indexed by `cam_id` and timestamp allow for efficient time-range queries. This is the sole metadata store for video segment indexing.

Storage Backend and Upload Strategy

Encoded segments are uploaded to MinIO (S3-compatible object storage) by default. This is the preferred storage backend, especially for on-prem or self-hosted deployments due to:

- Cost-efficiency
- S3 API compatibility
- Support for erasure coding and scalability For cloud-native deployments, AWS S3 Standard is supported as a backend. However, due to its high cost for frequent read/write and lifecycle operations, MinIO is recommended wherever possible, including in cloud VMs or Kubernetes clusters. The data layer is enhanced with a Redis Cluster for session management and real-time data, ClickHouse alongside TimescaleDB for analytics, and a distributed MinIO setup for scalable object storage.

Retention and Archival Policy

- **Hot Storage:** 24-hour playback segments remain in MinIO or S3 Standard.
- **Archival:** Lifecycle rules transition older segments to:
 - S3 Glacier Instant Retrieval / IA (cloud)
 - On-prem archival tier via MinIO versioning or external Glacier-copy (rsync-based)
- **Retention Control:** Configurable per camera, event type, or organization:
 - Standard streams: 7 days
 - Motion-tagged streams: 30 days (policy-driven)
 - Critical alerts: Extended retention or offsite copy This ensures cost optimization and compliance with video retention mandates. Intelligent tiering automates footage movement to cheaper cloud storage, predictive analytics alerts on

storage needs, and multi-cloud support (AWS, Google Cloud, Azure) ensures redundancy.

II. Search and Playback

The Search and Playback Engine enables efficient time-based and event-based retrieval of recorded video segments. It interacts with TimescaleDB to resolve metadata and delivers the corresponding video streams securely to the frontend.

Search API

The search interface is exposed via a RestAPI service, which queries TimescaleDB using camera ID and time range filters. Supported query types include:

- Timeline-based search (by cam_id and timestamp)
- Event-tag-based filtering (e.g., motion, intrusion)
- Paginated historical query (e.g., 7 days of activity) The API avoids reliance on file paths or Redis and uses only TimescaleDB for all segment lookup operations.

Segment Resolution and Access

Based on the search results, the segment file path (from storage_path) is resolved to:

- A pre-signed URL from MinIO or S3, expiring after a fixed duration
- A streaming path via Nginx proxy, where direct access is not desirable This ensures secure and tokenized access to stored video content without exposing raw URLs in the frontend.

3. AI Service and Event Management

The AI Service and Event Management module is responsible for real-time detection of security events (e.g., intrusion, loitering, object classification), managing associated media (snapshots and clips), and dispatching event metadata and notifications to clients. The system is designed for distributed deployment across edge and central locations, ensuring flexibility, low-latency processing, and horizontal scalability.

I. Stream Routing and Inference Workflow

Stream Ingestion and Routing

The AI module begins with a **GStreamer/FFmpeg-based decoding pipeline**, which pulls H.264/H.265 streams from cameras. An **AI Router** dynamically determines the inference path—**edge-based** (on AI-capable cameras) or **central server-based**—based on metadata in the camera registry.

- For **standard IP cameras**, streams are decoded and routed internally to the inference pipeline.
- For **edge AI cameras**, detection is performed on-device, and only metadata (e.g., bounding boxes, object class, timestamp) is pushed via REST APIs.

Routing logic follows the **Strategy Pattern**, using pluggable handlers for ONVIF and proprietary protocols, allowing modular extension for future camera types.

Inference Pipeline Execution

On the server side, decoded frames are passed to a **Model Orchestrator**, which distributes frames across a **Kubernetes-managed GPU node pool**. Frames are processed using:

- **YOLOv8** for general object/person/vehicle detection.
- **OpenCV/OpenVINO** for optimized face detection or lightweight AI inference.
- **ONNX models** for custom use cases (e.g., smoke, helmet detection).

Frames are handled in batches using an **asyncio-based producer-consumer queue**, supporting high-throughput parallelism and horizontal scaling. Redis is integrated for real-time caching of AI results between the Model Orchestrator and Event Management. Apache Kafka complements the inference pipeline for event streaming. MLflow is implemented for AI model versioning and deployment management to support library updates.

II. Event Detection, Asset Generation, and Metadata Handling

Real-Time Event Processing

When a model detects an actionable event (e.g., intrusion or loitering), an **Event Manager** is triggered. The manager performs the following:

- **Snapshot capture** from the current frame using OpenCV.

- **Clip generation** using a **60-second rolling circular buffer** maintained by FFmpeg (in-memory). On event trigger, a pre-30s and post-30s window is extracted using ffmpeg -ss and -t parameters.
- Snapshots and clips are encoded as H.265 .mp4 files and stored in **MinIO (preferred)** or **AWS S3 Standard**. MinIO is cost-efficient, S3 support exists for cloud-native environments, but S3 is considered expensive for frequent operations.

Metadata Indexing

Each detected event is indexed into **TimescaleDB**, with structured metadata including:

- event_id
- cam_id
- timestamp
- event_type
- bbox or object_class (if applicable)
- snapshot_path
- clip_path
- acknowledgment_status

This metadata supports both timeline rendering and advanced event filtering in the client UI.

4. Non-Core Modules

4.1 Notification and Event Dispatcher

The Notification and Event Dispatcher is responsible for real-time delivery of alerts generated by the Alert Management Service. It ensures event prioritization, queue handling, and multi-channel dispatch to various clients, such as the Web UI, mobile apps, or external push services.

Workflow and Processing

Upon receiving an alert from the Alert Management Service, the event is published to a **Kafka topic**. The **Message Dispatcher** consumes the event, processes it via a **worker pool**, and classifies it by type, severity, and intended recipients. The dispatcher then routes the notification via:

- **WebSocket** to browser and desktop clients for real-time UI updates.
- Optional integrations for push/mobile notifications via RestAPI endpoints.

The system supports priority queues, ensuring that critical alerts are handled before informational ones. Kafka **partitioning** is used for horizontal scalability, enabling parallel processing across alert categories or zones.

Features and Optimizations

- Real-time, multi-user notification dispatch using WebSocket
- Support for priority-based queuing and throttling
- Worker pool concurrency ensures high throughput
- Configurable delivery thresholds per user, site, or organization

4.2 Admin Services

The Admin Services module manages user roles, device configurations, site definitions, and server metadata for both Vigil-X sites and the Vigil-X center. It supports a multi-tenant structure with hierarchical customer and site relationships.

Workflow and Data Flow

All configuration changes and user requests from the frontend pass through a **RestAPI service**, which interacts with a **PostgreSQL database** for persistent storage. A **Redis cache** accelerates frequently accessed user sessions and configuration states. When changes occur (e.g., user role update or device edit), **WebSocket notifications** are triggered to synchronize UI state in real-time across all logged-in clients.

Features and Optimizations

- Role-based access control and single sign-on support
- Multi-time zone and multi-language configuration
- Site and server setup, including on-prem/cloud failover
- Redis caching to reduce DB load and latency
- WebSocket sync for live configuration updates across clients

4.3 Auto-Discovery Service

The Auto-Discovery Service is responsible for detecting and cataloging network-connected devices, including IP cameras, NVRs at each Vigil-X site. It enables seamless onboarding of new devices into the VMS ecosystem.

Discovery Workflow

The discovery engine initiates **parallel subnet scans** using the **ONVIF protocol**, along with **ARP**, **ISAPI** and **SSDP** methods for comprehensive network coverage. Discovered devices are classified using their capabilities, and hardware/firmware metadata is extracted.

The **Inventory Manager** stores these profiles temporarily and forwards validated device metadata to the **Vigil-X Media Server**. Device records are then indexed into **PostgreSQL**, enabling time-based activity tracking and visual mapping.

Features and Optimizations

- ONVIF-compliant scanning with fallback to ARP/SSDP
- Support for parallel scanning across VLANs/subnets
- Caching of scanned devices (using Redis) to avoid duplication
- Asset classification with specs, firmware, and connectivity metadata

4.4 Audit and Monitoring

The Audit and Monitoring module provides visibility into system activity, service health, and user actions across the platform. It aggregates logs and metrics from all major modules and displays them via customizable dashboards.

Logging and Monitoring Pipeline

Each component in the system sends logs to **Loki** and metrics to **Prometheus**, where they are tagged and grouped. These are visualized via **Grafana dashboards**, accessible to administrators. API integration via **RestAPI** exposes query endpoints for audit logs and SLA metrics.

Monitoring Agents run on each service node (both at site and center), and report health, performance, and uptime.

Features and Optimizations

- Real-time dashboards for camera status, AI alerts, recording health, etc.
- Detailed user audit trails (login, config changes, escalations)
- Distributed Prometheus setup ensures scalability with Kubernetes
- RestAPI query layer for metrics and audit retrieval

4.5 Health Monitoring

Health Monitoring ensures that all critical services, servers, and network paths across the Vigil-X system are operational and within SLA thresholds.

Health Check Architecture

Each node runs a **Health Checker** that periodically verifies the availability of services (Media Server, AI Engine, DBs, etc.). Results are pushed to **Prometheus**, where alert rules are evaluated. Health status is temporarily cached in **Redis** to enable fast access by Admin Services.

If a component fails, an **Alert Trigger** signals the Admin Dashboard and can escalate to system-level alerts.

Features and Optimizations

- Configurable service polling interval
- Redis-based health snapshot cache
- SLA-based alert prioritization
- Support for failover awareness and recovery tracking

4.6 Authentication (Auth Service)

The Auth Service handles secure login, token issuance, and role-based access enforcement across Vigil-X's distributed platform. It is used by all services and clients to validate and authorize requests.

Authentication Flow

When a client authenticates, credentials are validated by the **Authenticator**, and an **OAuth2/JWT-based token** is issued. This token is used for stateless API access. Session state and refresh tokens are temporarily stored in **Redis** for performance.

Role-based permissions are enforced by the **Role Manager**, which checks the user's access level for the requested operation (e.g., view-only, admin, AI-review).

HashiCorp Vault manages secrets, API rate limiting is implemented with Redis, a Web Application Firewall (WAF) is placed before NGINX, and Let's Encrypt automates certificate management.

Features and Optimizations

- OAuth2-compliant login and token refresh
- Stateless access via JWT tokens
- Multi-factor authentication support
- Role/permission mapping integrated with Admin Service
- Redis-backed session storage for fast validation

4.7 Password and Secrets Management

The Vigil-X ensures secure password and secrets management using HashiCorp Vault for both cloud and on-premises deployments, storing JWT signing keys, API secrets, OAuth credentials, and dynamic PostgreSQL credentials. Passwords are hashed with Argon2id and stored in PostgreSQL with encryption at rest (AWS KMS or pgcrypto), using per-user salts embedded in the hash. Vault enables audit logging, key rotation (every 90 days), and access control via AppRole and token policies. Redis caches session tokens with 1-hour TTLs, and PgBouncer optimizes PostgreSQL connections.

4.8 Event Management

The Event Management component processes video streams to detect, analyze, and store events, enabling real-time notifications and efficient event-based search. It operates within the Media Server at each site to ensure low-latency processing and independence from centralized services.

Stream Processing and Event Detection

A secondary/AI stream from the camera is processed by the Media Server, which hosts the AI Engine (YOLOv8, OpenVINO, or ONNX). The AI Engine analyzes the stream to detect events such as intrusion, motion, or objects, generating metadata including event type, confidence score, and timestamps.

Event Asset Generation

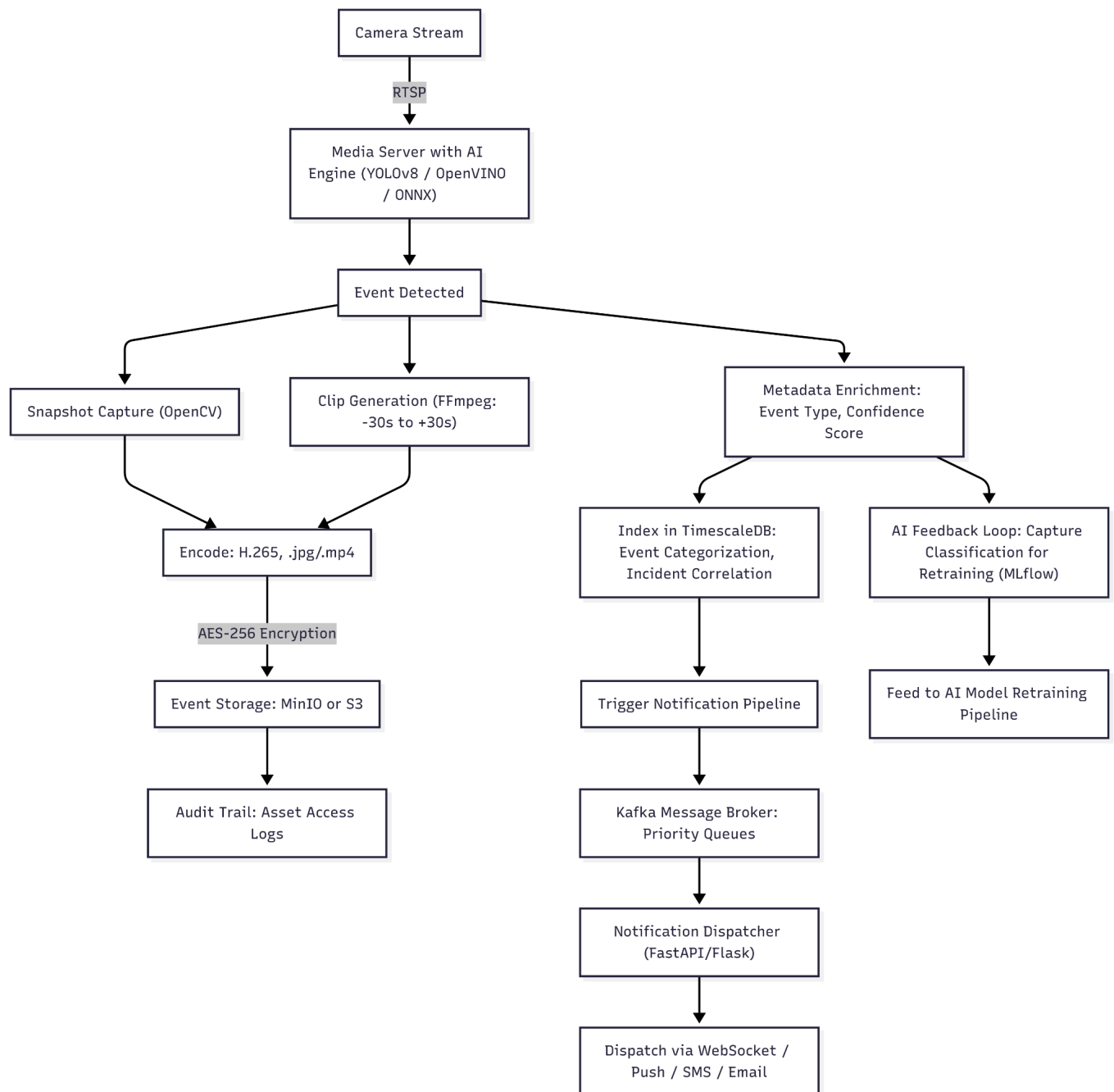
Upon event detection, the system captures a snapshot using OpenCV and generates a 60-second video clip (± 30 seconds around the event) using FFmpeg. Both snapshots and clips are encoded with H.265 in .jpg or .mp4 formats, respectively. The metadata is enriched with event categorization and incident correlation data to support linking events across cameras.

Storage and Indexing

Snapshots and clips are encrypted with AES-256 and stored in hot storage (MinIO or S3). Access to these assets is logged in an audit trail to ensure compliance. Event metadata, including camera ID, timestamps, event type, and confidence scores, is indexed in TimescaleDB for efficient search and retrieval. A feedback loop captures classification results for AI model retraining via MLflow.

Notification Pipeline

Indexed events trigger the notification pipeline through a Kafka message broker with priority queues. The Notification Dispatcher, built with FastAPI or Flask, processes these events and dispatches notifications via WebSocket, push, SMS, or email, ensuring real-time, multi-platform alerting.



4.9 Communication with External Devices

Event from camera to NVR/MediaServer comes with MQTT or HTTP(s)/websocket or Onvif . It depends on the type of camera we procure. Kafka is not possible to interface with camera events.

Implementation: Vigil-X uses MQTT for event-driven communication with devices like fire alarms, access control, or IoT sensors.

Details:

- **MQTT:** Offers lightweight messaging for IoT devices, enabling two-way communication (e.g., alarm status updates).
- **Workflow:** The Notification Dispatcher links device events to camera feeds, driving actions like PTZ focus or SMS alerts.

Example: A fire alarm sends an MQTT message to Vigil-X, which Kafka forwards to the Notification Dispatcher. Vigil-X then focuses nearby cameras and sends SMS alerts.

4.10 Handling External VMS Systems

- **Implementation:** Vigil-X integrates with external VMS (e.g., Genetec, Milestone) using ONVIF Profiles S/G/T, RTSP streaming, and REST APIs/webhooks for event sharing.
- **Details:**
 - **ONVIF Profiles:**
 - Profile S: Streams video from external VMS to Vigil-X.
 - Profile G: Enables search and playback of external VMS recordings.
 - Profile T: Shares analytics metadata, like motion alerts.
 - **REST APIs/Webhooks:** Sync alerts and metadata (e.g., intrusion events) between systems.
 - **Auto-Discovery:** ONVIF/SSDP finds external VMS devices and maps them to Vigil-X workflows.
- **Example:** A client with Genetec at Site A and Vigil-X at Sites B–D uses ONVIF Profile S to stream Genetec feeds to Vigil-X, with REST APIs syncing motion alerts to a shared dashboard.

WebRTC Security Summary

WebRTC (Web Real-Time Communication) is a powerful technology that enables peer-to-peer communication between browsers and mobile with SSL Certificates

1. Encryption

- **End-to-End Encryption (E2EE):** WebRTC supports encryption of both signaling and media streams to protect data in transit. Audio, video, and data channels are encrypted using standards like **DTLS (Datagram Transport Layer Security)** for data channels and **SRTP (Secure Real-Time Transport Protocol)** for media streams.
- **SRTP:** It ensures the confidentiality, integrity, and authentication of audio and video streams. No Playback possible. Each track encrypted with SSL certificates and fingerprints
- **DTLS:** Protects the signaling data between peers, preventing interception or tampering.

2. Authentication & Identity

- **ICE (Interactive Connectivity Establishment):** Ensures that WebRTC connections are established securely by handling NAT traversal and identifying the most secure path for communication. It also verifies that each peer is who it claims to be.
- **STUN (Session Traversal Utilities for NAT) and TURN (Traversal Using Relays around NAT):** These protocols help establish connectivity between peers, even in restrictive network environments. TURN, in particular, ensures that communication can occur securely when direct peer-to-peer connection isn't possible

3. Access Control

- **Permission Requests:** WebRTC requires explicit permission from the user to access hardware devices like microphones and cameras. Browsers ensure that users are informed when a website requests such access.
- **Secure Origin Policy:** WebRTC requires that websites using it be served over HTTPS (secure HTTP), preventing man-in-the-middle (MITM) attacks.

4. Signaling Security

- While WebRTC itself does not define a signaling protocol, it's important to ensure that any signaling communication (e.g., exchanging SDP data and ICE candidates) is done over a secure channel, typically using **WSS (WebSocket Secure)** or **HTTPS**. This helps prevent interception or tampering.

5. Privacy Concerns

- **IP Leaks:** WebRTC can potentially expose a user's local IP address even if they are behind a VPN or using other privacy mechanisms. This issue can be mitigated by ensuring that WebRTC is properly configured to limit such leaks.
- **Browser-Specific Risks:** Different browsers may have varying implementations of WebRTC security, which could introduce vulnerabilities. Users should keep their browsers up to date to minimize such risks.

6. Mitigating Attacks

- **Man-in-the-Middle (MITM):** Encryption (DTLS, SRTP) ensures that even if someone intercepts WebRTC traffic, it cannot be read or altered.
- **Denial-of-Service (DoS):** WebRTC uses ICE and TURN servers to ensure that communication remains stable even under potentially adverse conditions, but misconfigured or overloaded servers could be vulnerable to DoS attacks.
- **Eavesdropping & Replay Attacks:** The encryption mechanisms, coupled with integrity checks, protect against eavesdropping and replay attacks.

7. Regulatory Compliance of Indian government and System certificates

- WebRTC implementations may need to comply with privacy regulations (such as GDPR, HIPAA) if they are used for sensitive communications. Ensuring proper encryption and data handling policies is crucial for compliance.

Multi-Cloud Deployment

Goals

1. **High Availability:** Active-passive across cloud providers.
2. **Data Locality:** Media servers closer to camera/edge sources (sites).
3. **Cost Optimization:** Use S3/MinIO across providers with storage tiering.
4. **Regulatory Compliance:** Tenant-specific data residency enforcement.
5. **Disaster Recovery:** Cross-region replication for metadata and segments.

Architecture Options

1. Hub-and-Spoke Model
 - a. **Cloud Hub (VigilX-Center)** on AWS, Azure, or GCP
 - b. **Spoke deployments (VigilX-Site nodes)** hosted regionally or on-prem

- c. **Use STUN/TURN**, reverse tunnels, WebRTC and API Gateway for remote access
- 2. **Provider-Agnostic Deployment**
 - a. Use **Kubernetes + Helm + Terraform** to deploy same stack across:
 - i. AWS (EKS + S3)
 - ii. Azure (AKS + Blob Storage)
 - iii. GCP (GKE + Cloud Storage)
 - b. Media segments pushed to each provider's object store
 - c. API Gateway in each cloud for tenant-specific routing
- 3. **Failover Model**
 - a. One primary cloud handles traffic
 - b. Secondary remains in sync (replica DB, replica cache, standby media server etc.)
 - c. Automated failover using health probes and DNS TTL manipulation.
 - d. Use PostgreSQL write-replica with async replication.
 - e. Relay WebRTC sessions using TURN fallback in the passive zone if failover triggers.

Use Cases to Architecture Component Mapping

#	Use Case	Description	Architecture Components	Notes
1	Live Video Monitoring	View live streams with low latency	RTSP Ingest → WebRTC via Media Server → SFU → Web/Desktop/Mobile Clients (via STUN/TURN, ICE, SDP)	Supports NAT/firewall traversal with fallback
2	Playback of Old Recordings	Search and play recorded videos	Segmenter → TimescaleDB → S3/MinIO → Playback API → Secure Pre-signed URLs	Support for timeline and event filtering
3	Search by Timeline & Events	View timeline-based or event-tagged video	TimescaleDB → Search & Playback Service	REST API access for search queries
4	AI Events Based Search	Search recordings based on detected events	AI Inference Pipeline → Event Metadata in TimescaleDB → Filtered Search APIs	
5	Event Alerts	Get notified on	AI Engine → Event Manager → Kafka	Supports priority

	and Notifications	critical events in real-time	→ Notification Dispatcher → WebSocket & Push	queue and rate limiting
6	Device Auto-Discovery	Auto-detect new cameras/NVRs on the network	Auto-Discovery Service (ONVIF) → Inventory Manager → PostgreSQL	Supports parallel scan across virtual LANs
7	Health Monitoring	Track camera/server health and service status	Health Checker → Prometheus + Grafana Dashboards → Redis Cache	Support for SLA alerting
8	Role-Based Access Control (RBAC)	Restrict access based on user roles	Auth Service (OAuth2 + JWT) → Role Manager → Admin APIs	Support for hierarchical tenant-based access
9	Multi-Tenant Support	Isolated data and access control for each client	Admin Services → PostgreSQL Namespacing → Session Sync via WebSocket	Support for onboarding workflows
10	Audit Logging and Compliance	Track user actions, logins, config changes	Audit Service → Loki + Prometheus → Admin Dashboard → Queryable via API	Support for real-time and historical audit trail
11	Scalability & High Availability	Scale based on load, ensure no single points of failure	1. HAProxy Load Balancer 2. Kafka, Microservices, Docker+Kubernetes 3. Stateful Media Server	Support for scale-out and distributed zones.
12	Secure Access	End-to-end encrypted, NAT-safe video streaming	1. WebRTC with DTLS/SRTP (via ICE/STUN)	Encryption and identity enforced.
13	Web/Desktop/ Mobile UI Access	Use both browser and app to view/control system	1. Backend APIs via NGINX Gateway 2. WebRTC with STUN for live monitoring	

Relevant Links

[Question and answers doc:](#)

[Arch patterns doc:](#)

[Software Requirements Specification](#)

[Media server failure and handling](#)

- Include relevant question here..instead of links
- Prepare a module wise detailed document when we are working on that module
- Draw a for edge stream and non edge stream ,seperate